

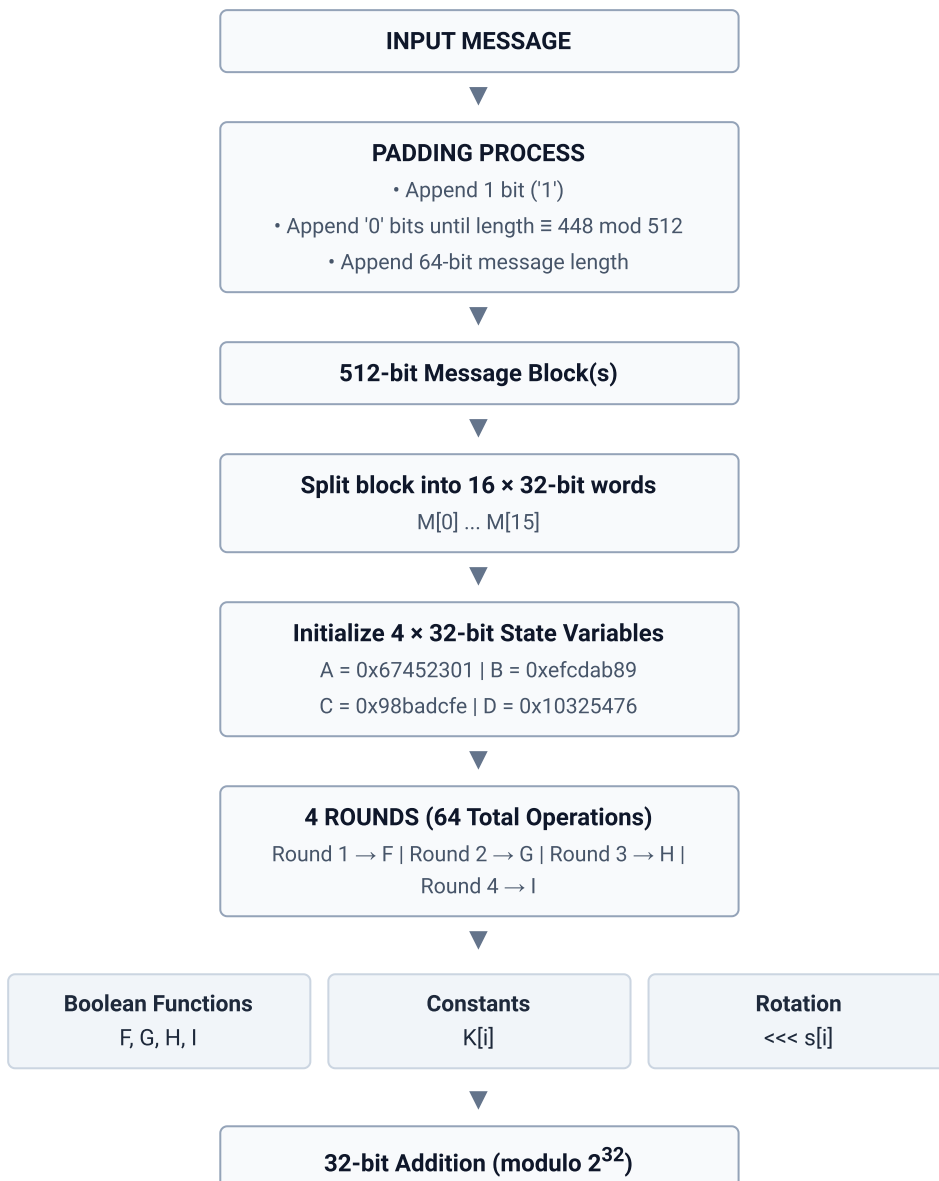
MD5 Operation Explained

1. OVERVIEW OF CRYPTOGRAPHIC HASH FUNCTIONS

A hash function is a deterministic algorithm that takes input data of arbitrary size (such as a message or file) and performs computations on it to produce a fixed-size bit string result called a **hash** or **digest**. A key property is that it always yields the identical output for the exact same input, whereas even a minuscule variation in the input produces a drastically different output.

Security Notice: MD5 produces a 128-bit (32 hexadecimal character) hash. MD5 is no longer considered collision-resistant because efficient methods exist to find two distinct inputs that yield the exact same hash (a collision). Consequently, MD5 must not be utilized for security-critical operations such as digital signatures or certificates.

2. HIGH-LEVEL DATA FLOW



3. DETAILED STEP-BY-STEP SUMMARY

- 1. Message Padding:** The input is padded by appending a single 1 bit followed by enough 0 bits so that the total bit length is 64 bits short of a multiple of 512. Finally, the original message length encoded as a 64-bit integer is appended, resulting in a message length exact to a multiple of 512 bits (one or several 512-bit blocks).
- 2. Word Decomposition:** Each 512-bit block is decomposed into 16 32-bit sub-blocks or "words", denoted as $M[0]$ through $M[15]$.
- 3. State & Lookup Table Initialization:** Four 32-bit state registers (A, B, C, D) are initialized with fixed standard hexadecimal constants:

Register	Initial Value (Hex)	Description
A	0x67452301	Word A state variable
B	0xefcdab89	Word B state variable
C	0x98badcfe	Word C state variable
D	0x10325476	Word D state variable

Two predefined lookup tables are also used:

- $K[i]$: Table of 64 constant values derived from the sine function.
- $s[i]$: Table specifying the per-operation left rotation bit shifts.

4. COMPRESSION ROUNDS & OPERATIONS

The core algorithm executes **64 operations** per 512-bit block, partitioned into 4 rounds of 16 operations each. Each round utilizes a distinct non-linear bitwise function:

Rounds 0–15 $\rightarrow F(B, C, D) = (B \text{ AND } C) \text{ OR } ((\text{NOT } B) \text{ AND } D)$ Rounds 16–31 $\rightarrow G(B, C, D) = (B \text{ AND } D) \text{ OR } (C \text{ AND } (\text{NOT } D))$ Rounds 32–47 $\rightarrow H(B, C, D) = B \text{ XOR } C \text{ XOR } D$ Rounds 48–63 $\rightarrow I(B, C, D) = C \text{ XOR } (B \text{ OR } (\text{NOT } D))$

Each individual operation consumes:

- One message word $M[g]$
- One constant $K[i]$
- One rotation shift amount $s[i]$
- One nonlinear function (F, G, H, or I)

5. MATHEMATICAL TRANSFORMATION & STATE UPDATE

The key mathematical step updates the state by calculating an intermediate value x , applying a circular left shift, and adding state variable B :

$$x = A + F(B, C, D) + M[g] + K[i] \quad \text{new_value} = B + \text{ROTATE_LEFT}(x, s[i])$$